

Basis Path Testing

White-Box Testing Technique

1 Introduction

Basis Path Testing is a white-box testing technique that ensures execution of all **independent paths** in a program at least once.

It was proposed by **Thomas J. McCabe** and is closely related to **Cyclomatic Complexity**.

The goal of basis path testing is to achieve **complete path coverage** with a minimum number of test cases.

2 Key Idea of Basis Path Testing

- A program is represented using a **Control Flow Graph (CFG)**
- The CFG is analyzed to identify all **independent paths**
- Each independent path is tested at least once

The number of independent paths is equal to the **Cyclomatic Complexity** of the program.

3 Independent Paths

An **independent path** is a unique path through the program that introduces at least one new edge not included in any other path.

- Independent paths form the **basis set**
- Executing all basis paths guarantees execution of every statement

4 Notation Clarification

To avoid confusion:

- P = Number of connected components in the CFG
- P_d = Number of predicate (decision) nodes

5 Relationship with Cyclomatic Complexity

Number of Basis Paths = Cyclomatic Complexity

This means:

- Cyclomatic Complexity provides the **upper bound** on the number of test cases
- Higher complexity $\Rightarrow$ more basis paths $\Rightarrow$ more testing effort

6 Steps in Basis Path Testing

1. Construct the **Control Flow Graph (CFG)** of the program
2. Compute the **Cyclomatic Complexity**
3. Identify the **basis set of independent paths**
4. Design test cases to execute each independent path

Executing all basis paths ensures **branch coverage** and **statement coverage**.

7 Example

Sample Code

```
if (x > 0)
    y = 1;
else
    y = 2;

print(y);
```

7.1 Control Flow Analysis

- One decision point (**if**)
- Two possible execution paths

7.2 Cyclomatic Complexity

$$V(G) = P_d + 1 = 1 + 1 = 2$$

7.3 Basis Paths

- Path 1: condition true $\rightarrow y = 1$
- Path 2: condition false $\rightarrow y = 2$

Two test cases are sufficient to fully test this program using basis path testing.

8 Worked Example: Basis Path Testing with Nested Conditionals

8.1 Code Fragment

```
int num1 = 6;
int num2 = 9;

if (num2 == 0) {
    cout << "num1/num2 is undefined" << endl;
} else {
    if (num1 > num2) {
        cout << "num1 is greater" << endl;
    } else {
        cout << "num2 is greater" << endl;
    }
}
```

8.2 Control Flow Graph (CFG)

8.3 Control Flow Analysis

The program contains:

- One outer decision: `if (num2 == 0)`
- One inner decision: `if (num1 > num2)`

Hence, the program has **nested conditional logic**.

8.4 Cyclomatic Complexity Calculation

Cyclomatic complexity can be computed using two common methods:

Method 1 (Edges–Nodes):

$$V(G) = E - N + 2P$$

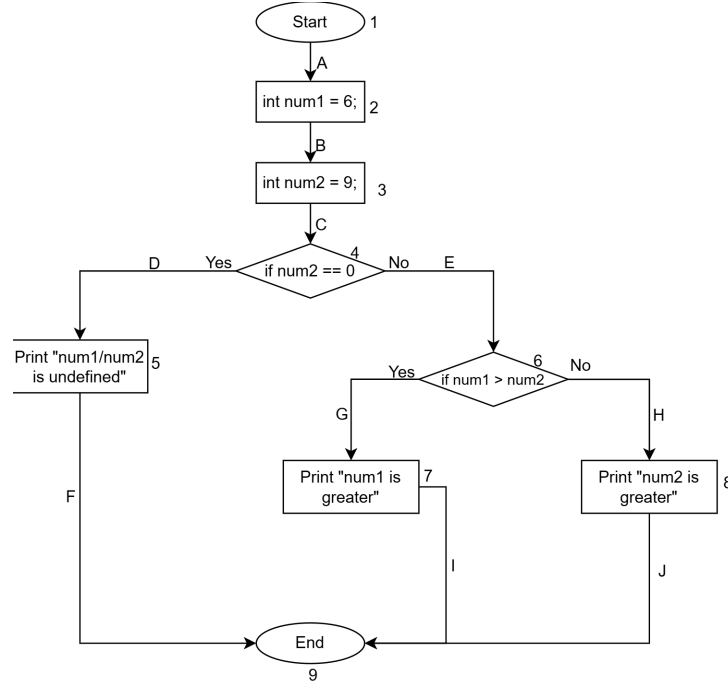


Figure 1: CFG for nested conditional example (two decision nodes: `num2==0` and `num1>num2`).

Method 2 (Predicate Nodes):

$$V(G) = P_d + 1$$

where:

- E = Number of edges in the Control Flow Graph
- N = Number of nodes in the Control Flow Graph
- P = Number of connected components in the graph (usually $P = 1$)
- P_d = Number of predicate/decision nodes (e.g., `if`, `while`, `for`)

Step 1: Identify Graph Parameters

From the CFG shown in the figure:

- Number of nodes: $N = 8$
- Number of edges: $E = 9$
- Number of connected components: $P = 1$
- Number of decision nodes: $P_d = 2$
 - `if (num2 == 0)`
 - `if (num1 > num2)`

Step 2: Compute $V(G)$ Using Both Methods

Using Method 1 (Edges–Nodes):

$$V(G) = E - N + 2P = 9 - 8 + 2(1) = 3$$

Using Method 2 (Predicate Nodes):

$$V(G) = P_d + 1 = 2 + 1 = 3$$

Therefore, the cyclomatic complexity of the program is:

$$V(G) = 3$$

This means there are **three independent execution paths** in the program.

8.5 Identification of Independent Paths

Based on the CFG, the independent paths are:

- **Path 1:** $1A \rightarrow 2B \rightarrow 3C \rightarrow 4D \rightarrow 5F \rightarrow 9$
(Case: $\text{num2} = 0$)
- **Path 2:** $1A \rightarrow 2B \rightarrow 3C \rightarrow 4E \rightarrow 6G \rightarrow 7I \rightarrow 9$
(Case: $\text{num2} \neq 0 \wedge \text{num1} > \text{num2}$)
- **Path 3:** $1A \rightarrow 2B \rightarrow 3C \rightarrow 4E \rightarrow 6H \rightarrow 8J \rightarrow 9$
(Case: $\text{num2} \neq 0 \wedge \text{num1} \leq \text{num2}$)

Each independent path introduces at least one new edge in the CFG.

8.6 Design of Test Cases

To execute all independent paths, the following test cases are required:

Path	Execution Path	num1	num2
Path 1	1A–2B–3C–4D–5F–9	9	0
Path 2	1A–2B–3C–4E–6G–7I–9	4	2
Path 3	1A–2B–3C–4E–6H–8J–9	6	8

These three test cases are sufficient to achieve complete basis path coverage.

8.7 Testing Interpretation

- Total independent paths = 3
- Minimum test cases required = 3
- All decisions and statements are executed at least once

This example demonstrates how nested conditionals increase cyclomatic complexity and testing effort.

9 Advantages of Basis Path Testing

- Ensures complete logical coverage
- Detects errors early in development
- Provides a systematic testing approach
- Helps identify unreachable code

10 Limitations of Basis Path Testing

- Becomes impractical for very large programs
- Does not test data-related errors
- Requires access to source code
- Focuses only on control flow, not functionality

11 Comparison with Other Testing Techniques

- **Black-box testing:** Focuses on input-output behavior
- **Basis path testing:** Focuses on internal logic and paths

Basis path testing is a **white-box testing technique**.

12 Key Exam Points

- Basis path testing ensures execution of all independent paths
- Number of test cases = Cyclomatic Complexity
- Uses Control Flow Graph
- Proposed by McCabe